

Работа с файлами

Потоковый ввод stdin и вывод stdout

Потоковый ввод `stdin`

Поток ввода в Python (**`sys.stdin`**) — это специальный объект в программе, куда попадает весь текст, который ввёл пользователь.

Потоком его называют потому, что данные хранятся в нем до тех пор, пока программа их не прочитала.

Таким образом, данные поступают в программу и временно сохраняются в потоке ввода (**`sys.stdin`**), а программа может забрать их оттуда, например, с помощью встроенной функции `input()`.

Потоковый ввод `stdin`

Поток ввода (`sys.stdin`) — является **итератором**, который невозможно перезапустить. Как и любой итератор, он может двигаться только вперёд.

Как только данные прочитаны, они удаляются из потока ввода безвозвратно.

Элементы, которые выдает этот итератор — это строки, введённые пользователем. Если пользовательский ввод закончен, то итератор прекращает работу.

Пока пользователь не ввёл последнюю строку, мы не знаем, сколько элементов в итераторе.

Потоковый ввод `stdin`

Чтобы работать с потоком ввода (`sys.stdin`), необходимо подключить модуль **sys** стандартной командой **import sys**.

Напишем программу, которая дублирует каждую введенную пользователем строку.

```
import sys
for line in sys.stdin:
    print(line.strip('\n'))
```

Строковый метод `strip('\n')` отрезает от строки `line` символ перевода строки, поскольку функция `print` сама переводит строку.

ПОТОКОВЫЙ ВВОД stdin

```
import sys
for line in sys.stdin:
    print(line.strip('\n'))
```

Пока есть данные в потоке ввода `sys.stdin` (то есть пока пользователь их вводит) программа будет записывать вводимые строки в переменную `line`, убирать символы перевода строки и выводить их на печать.



Если запустить такую программу, то она будет работать вечно.

ПОТОКОВЫЙ ВВОД stdin

```
import sys
for line in sys.stdin:
    print(line.strip('\n'))
```

Чтобы показать, что ввод закончен, недостаточно нажать Enter — компьютер не знает, завершил ли пользователь работу или будет ещё что-то вводить (при этом Enter превратится в пустую строку).



Для завершения ввода
необходимо ввести **Ctrl+D**

Потоковый ввод stdin. Чтение входных данных в список

С помощью потока ввода (sys.stdin) прочитать весь пользовательский ввод в список.

Реализуем с помощью генератора списка:

```
import sys
data = [line.strip() for line in sys.stdin]
print (data)
```

Протокол ввода:

10

десять

ten

Протокол вывода:

['10', 'десять', 'ten']



Сколько будет
введено строк
заранее не
известно!

Потоковый ввод stdin. Чтение входных данных в список

Реализуем с помощью функции высшего порядка `map()`:

```
import sys
data = list(map(str.strip, sys.stdin))
print (data)
```

Протокол ввода:

2

three

Протокол вывода:

['2', 'three']



Раньше приходилось в задачах сначала указывать количество строк, а затем вводить строки!

Потоковый ввод stdin. Чтение входных данных в список

Реализуем с помощью метода `readlines()`:

```
import sys
data = sys.stdin.readlines()
print (data)
```

Протокол ввода:

1
2
3
4

Протокол вывода:

```
['1\n', '2\n', '3\n', '4\n']
```



Метод `readlines()`
сохраняет символы
перевода строки

Потоковый ввод stdin. Чтение входных данных в текстовую переменную

Считать многострочный текст из стандартного потока ввода в текстовую переменную можно с помощью метода `read()`:

```
import sys
data = sys.stdin.read()
print (data)
print (type(data))
```

Протокол ввода:

Десять

Девять

Восемь

Семь

Протокол вывода:

Десять

Девять

Восемь

Семь

<class 'str'>

ПОТОКОВЫЙ ВЫВОД

Аналогичным образом можно работать с потоковым выводом (`sys.stdout`). По умолчанию функция `print()` перенаправляет вывод данных именно в `sys.stdout`, хотя нам ничего не мешает самостоятельно писать в него.

Приведенный ниже код:

```
import sys
print('Hello')
sys.stdout.write('world! ')
print('from')
sys.stdout.write('python\n')
print('Bye-bye')
```

Выводит:

```
Hello
world!from
python
Bye-bye
```



Функция `print()` добавляет перевод на новую строку, а явная запись данных в `sys.stdout` с помощью метода `write()` нет.

ПОТОКОВЫЙ ВЫВОД

Также при использовании потока вывода `sys.stdout` нужно самостоятельно преобразовывать данные к строковому типу данных (функция `print()` это делает автоматически).

Приведенный ниже код:

Исправленная версия кода:

```
import sys
sys.stdout.write(17)
```

```
import sys
sys.stdout.write(str(17))
```

приводит к возникновению ошибки.

Протокол вывода:

```
TypeError: descriptor 'encode' for 'str'
objects doesn't apply to a 'int' object
```

Примечания

Примечание 1. По умолчанию функция `input()` читает данные из потока ввода `sys.stdin`, а функция `print()` печатает данные в поток вывода `sys.stdout`.

Примечание 2. Функция `print()` — это удобная обертка (wrapper) вокруг метода `sys.stdout.write()`. Функцию `input()` часто можно рассматривать как обертку (wrapper) вокруг `sys.stdin.readline()`.

Примечания

Примечание 3. Объекты `sys.stdin` и `sys.stdout` являются файловыми объектами, предоставляемыми ОС. Когда программа запускается в интерактивном сеансе, `stdin` является клавиатурным вводом, а `stdout` является выводом на экран, но оболочка может использоваться для перенаправления из обычных файлов или вывода на канал и ввода в другие программы.

Примечания

Примечание 4. Приведенный ниже код:

```
import sys
temp = sys.stdout
sys.stdout = open('log.txt', 'w')
print('testing123')
print('another line')
sys.stdout.close()
sys.stdout = temp
print('back to normal')
```

приведет к созданию текстового файла **log.txt**,
содержащего:

```
testing123
another line
```

Работа с csv файлами

Формат CSV

CSV (от англ. Comma-Separated Values — значения, разделённые запятыми) — текстовый формат, предназначенный для представления табличных данных.

Строка таблицы соответствует строке текста, которая содержит одно или несколько полей, разделённых запятыми.

Формат CSV

Например, таблица:

Rank	Language	Share
1	Python	31.17%
2	Java	17.75%
3	JavaScript	8%
4	C#	7.05%
5	PHP	6.09%

в формате csv будет выглядеть так:

Rank , Language , Share

1 , Python , 31.17%

2 , Java , 17.75%

3 , JavaScript , 8%

4 , C# , 7.05%

5 , PHP , 6.09%



Пробелов после запятой
быть не должно

Работа с csv файлами

Рассмотрим текстовый файл **products.csv**, содержащий информацию о товарах некоторого интернет магазина. Файл содержит информацию о трех столбцах:

- words (ключевые слова)
- price (цена)
- product_name (имя продукта)

и имеет следующее содержимое:

```
words,price,product_name
Садовый стул,1699,ВЭДДО
Садовый стул,2999,ЭПЛАРО
Садовый стол,1999,ТЭРНО
Складной стол,7499,ЭПЛАРО
```



Разделитель записей - символ **НОВОЙ строки**, а разделитель полей — символ **запятой**.

Работа с csv файлами

Пример1. Приведенный ниже код:

```
with open('products.csv',  
          encoding='utf-8') as file:  
    data = file.read()  
    for line in data.splitlines():  
        print(line.split(','))
```

Используется
для
построчного
разделения
текста

ВЫВОДИТ:

```
['words', 'price', 'product_name']  
['Садовый стул', '1699', 'ВЭДДО']  
['Садовый стул', '2999', 'ЭПЛАРО']  
['Садовый стол', '1999', 'ТЭРНО']  
['Складной стол', '7499', 'ЭПЛАРО']
```

Работа с csv файлами

Пример2. Используем генератор списков для создания вложенных списков:

```
with open('products.csv',  
          encoding='utf-8') as file:  
    data = file.read()  
    table = [r.split(',') for r in  
              data.splitlines()]  
    del table[0]  
    print(table)
```

Удаляем заголовок

Получим:

```
[['Садовый стул', '1699', 'ВЭДДО'],  
 ['Садовый стол', '1999', 'ТЭРНО'],  
 ['Садовый стул', '2999', 'ЭПЛАРО'],  
 ['Складной стол', '7499', 'ЭПЛАРО']]
```

Работа с csv файлами

С помощью такой таблицы мы можем обратиться к цене седьмого по счету товара:

```
print(table[7][1])
```

Отсортировать товары по цене и напечатать 2 самых дешевых товара:

```
with open('products.csv', encoding='utf-8') as file:  
    data = file.read()  
    table = [r.split(',') for r in data.splitlines()]  
    del table[0]  
    table.sort(key=lambda x: int(x[1]))  
    for line in table[:2]:  
        print(line)
```

Работа с csv файлами



Какой символ в файле `products.csv` использовался в качестве разделителя полей?

В приведенном выше файле `products.csv` символ запятой использовался в качестве разделителя полей.

Однако бывают ситуации, в которых сам символ запятой является легитимным символом.

Например, столбец `words` может содержать (и как правило на практике содержит) несколько ключевых слов. В таком случае структура файла нарушается.

Работа с csv файлами

1) Если поле содержит запятые, то это поле должно быть заключено в двойные кавычки. Если этого не сделать, то данные невозможно будет корректно обработать.

Содержимое файла products.csv может иметь вид:

```
words,price,product_name  
"Садовый стул, стул для дачи",1699,ВЭДДО  
"Складной стол, обеденный стол",7499,ЭПЛАРО  
Стеллаж,1299,ХИЛЛИС  
"Кружка, сосуд, стакан с ручкой",39,СТЕЛЬНА
```



При таком содержимом файла products.csv код, написанный выше, является нерабочим, так как он разделит каждую строку через символ запятой.

Работа с csv файлами

2) Использовать в качестве разделителя другой символ, например, символ табуляции (\t), который весьма редко встречается в качестве валидного содержимого файла.

Содержимое файла products.csv может иметь вид:

```
keywords    price product_name
Садовый стул, стул для дачи 1699 ВЭДДО
Складной стол, обеденный стол 7499 ЭПЛАРО
Стеллаж      1299 ХИЛЛИС
Кружка, сосуд, стакан с ручкой 39 СТЕЛЬНА
```



Формат данных csv, в котором разделителем является символ табуляции, называют tsv (англ. tab separated values — «значения, разделенные табуляцией»).

Работа с csv файлами

tsv и **csv** — формы более общего формата **dsv** (англ. delimiter separated values — «значения, разграниченные разделителем»).



Приведенный выше код правильно обрабатывает файл `products.tsv`, в котором символом разделителя выбран символ табуляции (`\t`), если символ табуляции указан аргументом метода `split()`

Модуль csv

Несмотря на то что csv формат очень прост и мы можем работать с ним, как с обычным текстовым файлом, на практике используется встроенный модуль **csv**.

В данном модуле есть два основных объекта: **reader** и **writer**, созданные, чтобы читать и создавать csv файлы соответственно.

Чтение данных с помощью reader

Для импорта модуля мы используем строку кода:

```
import csv
```

Рассмотрим тот же файл products.csv:

```
import csv
a='products.csv'
with open(a, 'r', encoding='utf-8') as file:
    rows = csv.reader(file)
    for row in rows:
        print(row)
```

Создаем reader объект

Чтение данных с помощью reader

Объект **reader** дает доступ к построчному итератору, полностью аналогичному работе с файлом или списком.

После выполнения строки:

```
rows = csv.reader(file)
```

в переменную `rows` будет записан итератор, с помощью которого можно «пробежаться» циклом по файлу.

В каждой итерации цикла при этом будет доступна соответствующая строка файла, уже разбитая по запятым и представляющая собой список. При этом автоматически будут учтены все нюансы с запятыми внутри кавычек и самими кавычками.

Чтение данных с помощью reader

Например:

```
import csv
a='products.csv'
with open(a, 'r', encoding='utf-8') as file:
    rows = csv.reader(file)
    for words, price, product_name in rows:
        print(f'Ключи: {words}, цена:{price}')
```

использует распаковку списка и выводит:

```
Ключи: words, цена:price
Ключи: Садовый стул, цена:1699
Ключи: Садовый стул, цена:2999
Ключи: Садовый стол, цена:1999
Ключи: Складной стол, цена:7499
```

Чтение данных с помощью reader

При создании **reader** объекта мы можем его настраивать, указывая:

- ❑ аргумент **delimiter** — односимвольная строка, используемая для разделения полей, по умолчанию имеет значение ','
- ❑ аргумент **quotechar** — односимвольная строка, используемая для кавычек в полях, содержащих специальные символы, по умолчанию имеет значение '“'

Чтение данных с помощью reader

Пусть содержимое файла products.csv имеет вид (в качестве разделителя выбран символ ';'):

```
words;price;product_name
"Садовый стул, стул для дачи";1699;ВЭДДО
"Складной стол, обеденный стол";7499;ЭПЛАРО
Стеллаж;1299;ХИЛЛИС
"Кружка, сосуд, стакан с ручкой";39;СТЕЛЬНА
```

Чтение данных с помощью reader

Приведенный ниже код:

```
import csv
a='products.csv'
with open(a, 'r', encoding='utf-8') as file:
    rows = csv.reader(file, delimiter=';', \
quotechar='"')
    for index, row in enumerate(rows):
        if index > 2:
            break
        print(row)
```

выводит первые 3 строки файла, включая заголовки с названиями столбцов

Чтение данных с помощью DictReader

Использовать reader объект не всегда удобно, так как он возвращает сырые списки из строк файла, к тому же первой строкой является строка с названиями столбцов, которая практически всегда удаляется, так как мешает правильной обработке данных.

В модуле csv есть специальный объект DictReader, который поддерживает создание объекта-словаря на основе названий столбцов. С помощью DictReader объекта мы можем обращаться к полям не по индексу, а по названию, что делает код более понятным.

Чтение данных с помощью DictReader

Пусть для файла products.csv в качестве разделителя выбран символ ';'

Приведенный ниже код:

```
import csv
a='products.csv'
with open(a,'r',encoding='utf-8') as file:
    rows = csv.DictReader(file,
                          delimiter=';', quotechar='"')
    print(next(rows))
```

ВЫВОДИТ:

```
{ 'words': 'Садовый стул; стул для дачи', 'price':  
'1699', 'product_name': 'ВЭДДО' }
```

Чтение данных с помощью DictReader

При создании **DictReader** объектами значениями по умолчанию для аргументов `delimiter` и `quotechar` являются `,` (символ запятой) и `"` (символ двойной кавычки) соответственно.

При использовании **DictReader** не нужно избавляться от первой строки, содержащей названия столбцов.

При этом к элементам строк обращение теперь не по индексам, а по их названиям (`int(item['price'])`), что намного удобнее.

Чтение данных с помощью DictReader

Например:

```
import csv
a='products.csv'
with open(a,'r',encoding='utf-8') as file:
    rows = csv.DictReader(file)
    expensive = sorted(rows, key=lambda item:
                        int(item['price']), reverse=True)
    for record in expensive[:2]:
        print(record)
```

выводит 2 самых дорогих товара. Результат:

```
{'words': 'Складной стол, обеденный стол', 'price': '7499',
'product_name': 'ЭПЛАРО'}
{'words': 'Садовый стул', 'price': '2999', 'product_name':
'ЭПЛАРО'}
```

Запись данных с помощью writer

Для записи данных в csv файл можно использовать специальный writer объект.

```
import csv
columns = ['first_name', 'second_name',
           'number', 'letter']
data = [['Тимур', 'Гуев', 11, 'А'],
        ['Руслан', 'Чаниев', 9, 'Б']]
a='products.csv'
with open(a, 'w', encoding='utf-8', \
newline='') as file:
    writer = csv.writer(file)
    writer.writerow(columns)
    for row in data:
        writer.writerow(row)
```

Запись данных с помощью writer

создает файл students.csv с содержимым:

```
first_name,second_name,number,letter  
Тимур,Гуев,11,А  
Руслан,Чаниев,9,Б
```

Необязательный параметр **newline** функции `open()`, который имеет значение `"` (пустой строки) отвечает за переводы строк при чтении или записи в текстовый файл.

По умолчанию имеет значение `None`, в этом случае все разделители строк преобразуются в `'\n'`.

Если в файле оказывается лишний перевод строки, то следует использовать этот параметр в режиме `newline=""`, тогда `'\n'` будет преобразован в пустую строку.

Запись данных с помощью writer

При создании **writer** объекта мы так же можем его настраивать, задавая **delimiter** и многие другие параметры.

Например, если в вышеуказанном коде задать **writer** следующим образом:

```
writer = csv.writer(file, delimiter=';', \
quoting=csv.QUOTE_NONNUMERIC)
```

Получим:

```
"first_name";"second_name";"number";"letter"  
"Тимур";"Гуев";11;"А"  
"Руслан";"Чаниев";9;"Б"
```

Запись данных с помощью writer

Значение аргумента **quoting=csv.QUOTE_NONNUMERIC** означает, что в кавычки будут браться все нечисловые значения.

По умолчанию символом кавычки является `"`, если нужно поменять символ, то используйте именованный аргумент **quotechar**.

Запись данных с помощью writer

Для задания параметра **quoting** используются специальные константы из модуля csv:

QUOTE_ALL: указывает объектам записи указывать все поля

QUOTE_MINIMAL: указывает объектам записи заключать в кавычки только те поля, которые содержат специальные символы, такие как разделитель `delimiter`, кавычка `quotechar` или любой из символов в `lineterminator`

QUOTE_NONNUMERIC: указывает объектам записи указывать все нечисловые поля

QUOTE_NONE: указывает объектам записи никогда не заключать в кавычки поля

Запись данных с помощью writer

Помимо метода **writerow()** можно использовать и метод **writerows()**, чтобы записать сразу несколько строк.

Единственным аргументом этого метода может быть коллекция коллекций.

То есть, каждый элемент списка `rows` в нашем случае должен быть коллекцией. Если `rows` будет, например, списком чисел, программа завершится с ошибкой.

Запись данных с помощью writer

Приведенный ниже код:

```
import csv
columns = ['first_name', 'second_name',
           'number', 'letter']
data = [['Руслан', 'Чаниев', 9, 'Б'],
        ['Роман', 'Белых', 10, 'В']]
a='products.csv'
with open(a, 'w', encoding='utf-8', \
newline='') as file:
    writer = csv.writer(file,
                        delimiter=';',
                        quoting=csv.QUOTE_NONNUMERIC)
    writer.writerow(columns)
    writer.writerows(data)
```

Запись данных с помощью writer

создает файл students.csv с содержимым:

```
"first_name";"second_name";"number";"letter"  
"Руслан";"Чаниев";9;"Б"  
"Роман";"Белых";10;"В"
```

Запись данных с помощью writer

Для записи данных в csv файл также можно использовать DictWriter объект, который позволяет записывать содержимое словаря в файл.

```
import csv
data = [{'first_name': 'Тимур', 'second_name':
'Гуев', 'number': 11, 'letter': 'A'}]
columns = ['first_name', 'second_name', 'number',
'letter']
a='products.csv'
with open(a,'w',encoding='utf-8',\
newline='') as file:
    writer = csv.DictWriter(file,
        fieldnames=columns, delimiter=';',
        quoting=csv.QUOTE_NONNUMERIC)
    writer.writeheader()
    for row in data:
        writer.writerow(row)
```

Запись данных с помощью writer

создает файл students.csv с содержимым:

```
"first_name";"second_name";"number";"letter"  
"Тимур";"Гуев";11;"А"
```



Ключи словарей, которые записываются в файл, должны совпадать с названиями полей, которые переданы в качестве аргумента fieldnames, иначе будет возникать ошибка ValueError.

Запись данных с помощью writer

Можно использовать метод `writerows()` объекта `DictWriter` для записи сразу нескольких строк. Таким образом вместо строк кода:

```
for row in data:  
    writer.writerow(row)
```

МОЖНО НАПИСАТЬ:

```
writer.writerows(data)
```

Примечания

Примечание 1.

Достаточно распространенная ошибка при чтении данных из csv файла — это передача в `csv.reader` (`csv.DictReader`) не файлового объекта, а текстовых данных, считанных из файла. Это не вызывает ошибки, но результат будет отличаться от ожидаемого.

Примечания

Примечание 2. При создании reader объекта важно, чтобы файл был открыт в режиме чтения ('r'), аналогично при создании writer объекта — в режиме записи ('w').

Примечание 3. Так как reader объект является итератором, мы можем преобразовать его в список, кортеж или множество с помощью встроенных функций `list()`, `tuple()` и `set()` соответственно.

Работа с json файлами

Формат json

JSON (англ. JavaScript Object Notation) — текстовый формат обмена данными, основанный на языке программирования JavaScript.

Как и многие другие текстовые форматы, JSON легко читается людьми.

Несмотря на происхождение от JavaScript, формат считается независимым от языка и может использоваться практически с любым языком программирования.

Формат json

За счёт своей лаконичности формат JSON отлично подходит для **сериализации** сложных структур.

Преобразование переменных программы (Python-объектов) в формат для хранения называется «**сериализацией**», а обратное преобразование — «**десериализацией**».

Синтаксис json формата

Данные в формате json имеют структуру ключ-значение. Напоминает словарь Python, но ключи в json могут быть только **строками, заключенными в двойные кавычки**:

```
{  
  "firstName": "Тимур",  
  "lastName": "Гуев",  
  "gender": "мужской"  
}
```



В формате JSON
используются только
двойные кавычки!

Синтаксис json формата

Значениями в json формате могут быть не только строки. Это могут быть числа, списки значений, литералы true/false/null, а также вложенные объекты:

```
{  
  "firstName": "Тимур",  
  "lastName": "Гуев",  
  "age": 29,  
  "gender": "мужской",  
  "address": {  
    "streetAddress": "Часовая 25, кв. 127",  
    "city": "Москва",  
    "postalCode": 125315  
  },  
  "phoneNumbers": ["+7 (919) 424-84-34", "+7  
(916) 928-92-34"]  
}
```



Вложенность
данных может быть
бесконечной!

Модуль json. Функция dumps()

В Python для сериализации и десериализации в формат **json** есть модуль, который называется **json**.

Функция **dumps()** модуля **json** используется для сериализации данных в json строку. Для того, чтобы сериализовать данные с ее помощью, достаточно передать в нее аргументом любой сериализуемый Python объект.



Так как json — текстовый формат, то сериализация в него — это по сути преобразование данных в строку!

Модуль json. Функция dumps()

Приведенный ниже код:

```
import json
data = {'name': 'Russia', 'phone_code': 7,
        'capital': 'Moscow', 'currency': 'RUB'}
json_data = json.dumps(data)
print(type(json_data))
print(json_data)
```

ВЫВОДИТ:

```
<class 'str'>
{"name": "Russia", "phone_code": 7,
"capital": "Moscow", "currency": "RUB"}
```



В результирующую строку всегда попадают двойные кавычки!

Модуль json. Функция dump()

Функция dump() записывает переданный Python объект в файл.

```
import json
data = {'name': 'Russia', 'phone_code': 7,
        'capital': 'Moscow', 'currency': 'RUB'}
with open('countries.json', 'w') as file:
    json.dump(data, file)
```

создает файл countries.json и сохраняет в него информацию из словаря data в json формате.

Файл countries.json содержит одну строку без форматирования:

```
{"name": "Russia", "phone_code": 7,
"capital": "Moscow", "currency": "RUB"}
```

Необязательные аргументы функций **dumps()** и **dump()**

Функции записи **dumps()** и **dump()** имеют необязательные аргументы **indent**, **sort_keys** и **separators**.

Аргумент **indent** задает отступ от левого края. По умолчанию имеет значение **None** для более компактного представления без отступов.

Необязательные аргументы функций `dumps()` и `dump()`

Приведенный ниже код:

```
import json
data = { 'name': 'Russia', 'phone_code': 7,
        'capital': 'Moscow', 'currency': 'RUB' }

json_data1 = json.dumps(data, indent=2)
json_data2 = json.dumps(data, indent=10)

print(json_data1)
print(json_data2)
```

Необязательные аргументы функций

`dumps()` и `dump()`

ВЫВОДИТ:

```
{
  "name": "Russia",
  "phone_code": 7,
  "capital": "Moscow",
  "currency": "RUB"
}
{
  "name": "Russia",
  "phone_code": 7,
  "capital": "Moscow",
  "currency": "RUB"
}
```

Необязательные аргументы функций dumps() и dump()

Если значением `indent` является строка, то она используется в качестве отступа.

Приведенный ниже код:

```
import json
data = {'name': 'Russia', 'phone_code': 7,
        'capital': 'Moscow', 'currency': 'RUB'}
json_data = json.dumps(data, indent='++++')
print(json_data)
```

Необязательные аргументы функций

`dumps()` и `dump()`

ВЫВОДИТ:

```
{  
++++"name": "Russia",  
++++"phone_code": 7,  
++++"capital": "Moscow",  
++++"currency": "RUB"  
}
```



Отступов не будет, если значение аргумента **indent** равно 0, отрицательному числу или пустой строке.

Необязательные аргументы функций dumps() и dump()

Аргумент **sort_keys** задает сортировку ключей в результирующем **json**.

По умолчанию имеет значение **False** для более быстрого создания **json**.

Если установить значение аргумента в **True**, то ключи будут отсортированы в алфавитном порядке, что особенно удобно, когда ключей много.

Необязательные аргументы функций dumps() и dump()

Приведенный ниже код:

```
import json

data = {'name': 'Russia', 'phone_code': 7,
        'capital': 'Moscow', 'currency': 'RUB'}

json_data1 = json.dumps(data, indent=3)
json_data2 = json.dumps(data, indent=3,
                        sort_keys=True)

print(json_data1)
print(json_data2)
```

Необязательные аргументы функций

`dumps()` и `dump()`

ВЫВОДИТ:

```
{
  "name": "Russia",
  "phone_code": 7,
  "capital": "Moscow",
  "currency": "RUB"
}
{
  "capital": "Moscow",
  "currency": "RUB",
  "name": "Russia",
  "phone_code": 7
}
```

Необязательные аргументы функций dumps() и dump()

Аргумент **separators** задает кортеж, состоящий из двух элементов (**item_separator**, **key_separator**), которые представляют разделители для элементов и ключей.

По умолчанию аргумент имеет значение (' ', ': ').

Необязательные аргументы функций `dumps()` и `dump()`

Приведенный ниже код:

```
import json
data = {'name': 'Russia', 'phone_code': 7,
        'capital': 'Moscow'}
json_data = json.dumps(data, indent=3,
                        separators=(',', '='))
print(json_data)
```

ВЫВОДИТ:

```
{
    "name" = "Russia";
    "phone_code" = 7;
    "capital" = "Moscow"
}
```

Функция loads()

Для десериализации данных нужно использовать функцию `loads()`. Ее аргумент — это строка с данными в формате `json`.

Приведенный ниже код десериализует `json` строку в словарь:

```
import json
json_data = '{"name": "Russia",
"phone_code": 7, "capital": "Moscow"}'
data = json.loads(json_data)
print(type(data))
print(data)
```

Функция loads()

ВЫВОДИТ:

```
<class 'dict'>
{'name': 'Russia', 'phone_code': 7,
'capital': 'Moscow'}
```

В случае если строка для десериализации содержит данные с ошибкой, то модуль json не сможет правильно прочесть такую строку, и программа завершится с ошибкой:

json.decoder.JSONDecodeError

Функция load()

Функция load() принимает файловый объект и возвращает его десериализованное содержимое.

Пусть файл data.json имеет следующее содержимое:

```
{  
  "name": "Russia",  
  "phone_code": 7,  
  "capital": "Moscow",  
  "cities": ["Abakan", "Anadyr", "Anapa"],  
}
```

Функция load()

Приведенный ниже код:

```
import json
with open('data.json') as file:
    data = json.load(file)
    for key, value in data.items():
        if type(value) == list:
            print(f'{key}: {" ".join(value)}')
        else:
            print(f'{key}: {value}')
```

читает содержимое **data.json** файла в словарь **data** и выводит его содержимое:

```
name: Russia
phone_code: 7
capital: Moscow
cities: Abakan, Anadyr, Anapa
```

Изменение типа данных

При преобразовании данных в формат JSON, данные не всегда будут того же типа, что исходные данные в Python.

Например, кортежи при записи в JSON превращаются в списки.

Так происходит из-за того, что в JSON используются другие типы данных, и не для всех типов данных Python есть соответствия.

Изменение типа данных

Например:



Что получим?

```
import json
data = {
    'name': 'Russia',
    'timezones': ('Anadyr', 'Barnaul',
'Moscow', 'Kirov')
}
json_data = json.dumps(data)
new_data = json.loads(json_data)
print(data == new_data)
```

False

Изменение типа данных

Таблица конвертации типов данных Python в JSON:

Python	JSON
dict	object
list, tuple	array
str	string
int, float	number
True	true
False	false
None	null

Изменение типа данных

Таблица конвертации JSON в типы данных Python:

JSON	Python
object	dict
array	list
string	str
number (int)	int
number (real)	float
true	True
false	False

Ограничение по типам данных

В формат JSON нельзя записать словарь, у которого ключи – кортежи.

Приведенный ниже код:

```
import json
data = {
    'year': 2023,
    ('Timur', 'Guev'): 29,
    ('Arthur', 'Kharisov'): 20
}
json_data = json.dumps(data)
print(json_data)
```

генерирует ошибку:

`TypeError: keys must be str, int, float, bool or None, not tuple`

Ограничение по типам данных

С помощью необязательного аргумента `skipkeys` можно игнорировать ключи типа `tuple`.

Приведенный ниже код:

```
import json
data = {
    'year': 2023,
    ('Timur', 'Guev'): 29,
    ('Arthur', 'Kharisov'): 20
}
json_data = json.dumps(data, skipkeys=True)
print(json_data)
```

ВЫВОДИТ:

```
{"year": 2023}
```

Ограничение по типам данных

В JSON ключами словаря могут быть только строки. Но, если в словаре Python использовались числа, булевы значения или None, то они будут преобразованы в строки.

```
import json
data = {1: 'Timur', False: 'Arthur', None:
'Ruslan'}
json_data = json.dumps(data)
print(json_data)
```

ВЫВОДИТ:

```
{"1": "Timur", "false": "Arthur", "null":
"Ruslan"}
```

Кириллические символы в json

Приведенный ниже код:

```
import json
data = {'firstName': 'Тимур',
        'lastName': 'Гуев'}
s = json.dumps(data)
print(s)
```

ВЫВОДИТ:

```
{"firstName": "\u0422\u0438\u043c\u0443\u0440",
"lastName": "\u0413\u0443\u0435\u0432"}
```

Каждая буква из строк Тимур и Гуев заменена на ее код. Эти коды стандартны, и код для каждой из букв индивидуален. Например: 0438 — код буквы и.

Кириллические символы в json

Обратное преобразование из строки в словарь вернет закодированное значение в первоначальный вид:

```
import json
data = {'firstName': 'Тимур' }
s = json.dumps(data)
print(s)
result = json.loads(s)
print(result)
```

ВЫВОДИТ:

```
{"firstName": "\u0422\u0438\u043c\u0443\u0440" }
{'firstName': 'Тимур' }
```



Благодаря стандартным кодам, символы будут прочитаны и преобразованы в нужный вид любой программой на любом языке программирования.

Кириллические символы в json

С помощью необязательного аргумента `ensure_ascii` функций `dumps()` и `dump()` можно отказаться от такого кодирования.

```
data = {'firstName': 'Тимур'}  
s = json.dumps(data, ensure_ascii=False)  
print(s)  
result = json.loads(s)  
print(result)
```

```
{"firstName": "Тимур"}  
{ 'firstName': 'Тимур' }
```

ВЫВОДИТ:



Python преобразует такую строку обратно в словарь без проблем (поскольку использует Unicode по умолчанию), но это может привести к проблемам с преобразованием в программах, написанных на других языках программирования.

Примечания

Примечание 1. Можно сериализовать любой объект, поддерживаемый форматом json, например число, список, строку и т.д.

Приведенный ниже код:

```
import json
colors = ['white', 'red', 'black']
with open('list.json', 'w') as file:
    json.dump(colors, file, indent='---')
```

создает файл list.json со следующим содержанием:

```
[
---"white",
---"red",
---"black"
]
```

Примечания

Примечание 2. Регистр учитывается — имена с буквами в разных регистрах считаются разными. Повторяющиеся имена ключей допустимы, учитывается последний ключ.

Приведенный ниже код:

```
import json
data = {'firstName': 'Тимур',
        'lastName': 'Гуев', 'firstName':
        'Теймур', 'FirstName': 'Тим'}
s = json.dumps(data, ensure_ascii=False)
print(s)
```

ВЫВОДИТ:

```
{"firstName": "Теймур", "lastName":
"Гуев", "FirstName": "Тим"}
```

Примечания

Примечание 3. JSON обладает рядом преимуществ, которые и сделали его популярным:

- ❑ Не занимает много места, является компактным в написании и быстро компилируется
- ❑ Создание текстового содержимого понятно человеку, просто в реализации, а чтение со стороны среды разработки не вызывает никаких проблем. Чтение может осуществляться и человеком, поскольку ничего сложного в представлении данных нет
- ❑ Структура преобразуется для чтения на любых языках программирования
- ❑ Практически все языки имеют соответствующие библиотеки или другие инструменты для чтения данных JSON

Работа с zip файлами в Python

Формат zip

В реальной жизни применяется большое количество различных форматов архивов: zip, 7z, rar и многие другие. Рассмотрим zip архивы.

zip — это формат сжатия без потерь: после распаковки данные будут такими же, как перед сжатием. Алгоритм ищет избыточности в исходных данных и эффективнее представляет информацию.

Формат zip

Формат zip обладает следующими преимуществами:

- ☐ является полностью открытым
- ☐ является популярным (большинство архивов в Internet – это архивы zip)
- ☐ является очень быстрым

Модуль zipfile

В Python для работы с zip архивами используется встроенный модуль **zipfile**.

Основное преимущество данного модуля заключается в том, что он позволяет работать с архивом, как с обычной папкой, содержащей файлы и другие каталоги.

Модуль zipfile

Для того чтобы начать работать с zip архивами в Python, нам потребуется импортировать модуль **zipfile** и создать объект **ZipFile**.

```
from zipfile import ZipFile
```



Объекты **ZipFile** похожи на файловые объекты, возвращаемые функцией **open()**.

Метод `printdir()`

Будем работать со следующим [zip архивом](#)

Необходимо создать объект типа `ZipFile` и передать ему имя архива (архив лежит в той же папке, что и программа).

Метод **`printdir()`** выводит таблицу с информацией о содержимом архива:

- ❑ полные названия файлов с указанием даты изменения и размера в байтах.

Метод printdir()

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    zip_file.printdir()
```

выводит информацию о содержимом архива:

File Name	Modified	Size
test/	2021-11-27 12:47:10	0
test/Картинки/	2021-11-27 12:49:02	0
test/Картинки/1.jpg	2021-09-02 12:30:20	90156
test/Картинки/avatar.png	2021-08-20 09:38:44	19053
test/Картинки/certificate.png	2021-10-23 09:46:36	43699
test/Картинки/py.png	2021-07-28 17:55:56	33522
test/Картинки/World_Time_Zones_Map.png	2021-11-08 07:30:06	2324421
test/Картинки/Снимок экрана.png	2021-10-01 20:47:02	10878
test/Неравенства.djvu	2021-08-19 08:39:06	5283010
test/Программы/	2021-11-27 12:48:20	0
test/Программы/image_util.py	2021-11-18 12:42:22	4955
test/Программы/sort.py	2021-11-14 19:31:02	61
test/Разные файлы/	2021-11-27 12:48:10	0
test/Разные файлы/astros.json	2021-11-08 09:29:58	505

Метод `printdir()`

При создании объекта `ZipFile` мы также можем передать необязательный аргумент **`mode`**, который задает режим работы (по аналогии с обычными файлами):

- ❑ `r` — файл будет открыт для чтения
- ❑ `w` — если файл существует, то он будет уничтожен и вместо него будет создан новый файл
- ❑ `a` — существующий файл будет открыт в режиме добавления в конец



По умолчанию параметр `mode` имеет значение `'r'`.

Метод `infolist()`

Метод `infolist()` позволяет получить информацию о файлах из архива в виде списка специальных объектов (тип `ZipInfo`), которые содержат дополнительную информацию о каждом файле:

- ❑ **`file_size`** - размер начального файла в байтах
- ❑ **`compress_size`** - размер сжатого файла в байтах
- ❑ **`filename`** - имя файла
- ❑ **`date_time`** - дата изменения файла, представляет из себя кортеж: (год, месяц, день, час, минута, секунда).
- ❑ ...

Метод printdir()

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    info = zip_file.infolist()
    print(info[6].file_size)
    print(info[6].compress_size)
    print(info[6].filename)
    print(info[6].date_time)
```

выводит информацию о конкретном файле:

2324421

2322032

test/Картинки/World_Time_Zones_Map.png
(2021, 11, 8, 7, 30, 6)

Метод is_dir()

Метод is_dir() используется, чтобы проверить является ли объект папкой.

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    info = zip_file.infolist()
    print(info[0].is_dir())
    print(info[6].is_dir())
```

ВЫВОДИТ:

True

False

Метод `namelist()`

Метод `namelist()` возвращает список названий файлов и директорий, содержащихся в архиве.

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    info = zip_file.namelist()
    print(*info, sep='\n')
```

ВЫВОДИТ:

```
test/
test/Картинки/
test/Картинки/1.jpg
test/Картинки/avatar.png
test/Картинки/certificate.png
test/Картинки/py.png
test/Картинки/World_Time_Zones_Map.png
test/Картинки/Снимок экрана.png
test/Неравенства.djvu
test/Программы/
test/Программы/image_util.py
test/Программы/sort.py
test/Разные файлы/
test/Разные файлы/astros.json
```

Метод `getinfo()`

Метод `getinfo()` позволяет получить информацию о конкретном файле по его имени в архиве.

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    info = zip_file.namelist()
    last_file = zip_file.getinfo(info[-4])
    print(last_file.file_size)
    print(last_file.filename)
```

ВЫВОДИТ:

4955

test/Программы/image_util.py

Работа с конкретными файлами из архива

Рассмотрим обработку конкретного файл архива.

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    a='test/Разные файлы/astros.json'
    with zip_file.open(a) as file:
        print(file.read())
```

Открывает определенный файл в бинарном виде

Работа с конкретными файлами из архива

Символ `b` перед выводом указывает, что выведена бинарная строка.

ВЫВОДИТ:

```
b'{"number": 10, "people": [{"craft":  
"ISS", "name": "Mark Vande Hei"},  
{"craft": "Shenzhou 13", "name": "Wang  
Yaping"}, {"craft": "Shenzhou 13",  
"name": "Ye Guangfu"}], "message":  
"success"}'
```



Метод `file.read()` возвращает сырые байты (тип `bytes`).

Работа с конкретными файлами из архива

Метод **ZipFile.open()** открывает файл именно в бинарном виде, не в текстовом.

Чтобы преобразовать сырые байты в строку, нужно использовать метод **decode()**, указав нужную кодировку (например, UTF-8).

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    a='test/Разные файлы/astros.json'
    with zip_file.open(a) as file:
        print(file.read().decode('utf-8'))
```

Работа с конкретными файлами из архива

ВЫВОДИТ:

Символ `b` отсутствует

```
{"number": 10, "people": [{"craft":  
"ISS", "name": "Mark Vande Hei"},  
{"craft": "Shenzhou 13", "name": "Wang  
Yaping"}, {"craft": "Shenzhou 13",  
"name": "Ye Guangfu"}], "message":  
"success"}
```

Запись в zip архив

По аналогии с чтением файлов из архива их можно записывать в архив, для этого необходимо создать объект `ZipFile` в режимах `mode='w'` или `mode='a'`.

Для записи файла в архив используется метод **`write()`**, который принимает имя **существующего** файла.

Если файлы для записи в архив не будут найдены, то возникнет ошибка (исключение) **`FileNotFoundError`**.

Запись в zip архив

Приведенный ниже код:

```
from zipfile import ZipFile
with ZipFile('archive.zip', mode='w')
as zip_file:
    zip_file.write('program.py')
    zip_file.write('lse.jpeg')
    print(zip_file.namelist())
```

создает в папке с программой архив с именем archive.zip и записывает в него содержимое файлов program.py и lse.jpeg, которые так же находятся в папке с программой, а затем выводит список всех файлов данного архива.

Запись в zip архив

Метод `write()` может принимать еще один строковый аргумент, задающий новое имя файла в архиве.

```
from zipfile import ZipFile
with ZipFile('archive.zip', mode='w')
as zip_file:
    file = 'program.py'
    zip_file.write(file, 'new_prog.py')
    print(zip_file.namelist())
```

Выводит:

```
['new_prog.py']
```

Запись в zip архив

Для добавления файлов в уже существующий архив необходимо создать объект ZipFile в режиме mode='a'.

```
from zipfile import ZipFile
arh = 'test.zip'
with ZipFile(arh, mode='a') as zip_file:
    zip_file.write('pr.py', 'test/pr.py')
    zip_file.write('lse.jpeg')
    print(*zip_file.namelist(), sep='\n')
```

Приведенный код добавляет два новых файла. pr.py добавлен в папку test, lse.jpeg добавлен в корень архива. Затем распакован и выведен список всех файлов данного архива.

каталог

Если требуется извлечь отдельные файлы, то используется метод **extract()**, он принимает два аргумента: название файла и путь, по которому требуется извлечь файл.

Если путь не указывать, то файл будет извлечен в папку, где находится файл с программой.

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    zip_file.extract('test/avatar.png')
    zip_file.extract('lse.jpeg')
```

Приведенный код извлекает файлы из архива test.zip.

Извлечение содержимого zip-файла в каталог

Если требуется извлечь все содержимое архива, то используется метод **extractall()**, он принимает в качестве аргумента путь, по которому требуется извлечь все файлы.

```
from zipfile import ZipFile
with ZipFile('test.zip') as zip_file:
    zip_file.extractall()
```

Приведенный код извлекает все содержимое архива test.zip в папку, где находится файл с программой, при этом структура каталогов архива сохраняется.

Примечание.

Для того чтобы проверить является ли некоторый файл zip архивом, используется функция **zipfile.is_zipfile()**, которая принимает на вход путь к файлу (или сам файловый объект) и возвращает значение True, если указанный файл является zip архивом, или False в противном случае.

Робота с pickle файлами

Сериализация и десериализация

JSON — один из популярных форматов для сериализации, он текстовый, легковесный и легко читается человеком.

С помощью встроенного модуля `json` можно сериализовать встроенные типы данных: `bool`, `dict`, `int`, `float`, `list`, `string`, `tuple`, `None`

Помимо сериализации в формат `json` мы также можем использовать бинарную сериализацию, то есть сериализацию в байты. Для этого в Python используется модуль **`pickle`**.

Модуль pickle

Интерфейс взаимодействия с модулем pickle абсолютно такой же, как и для модуля json. Мы будем использовать четыре основных функции:

- ☐ dump()
- ☐ load()
- ☐ dumps()
- ☐ loads()

Функции `dump()` и `load()`

Функция `dump()` модуля `pickle` принимает сериализуемый Python объект, сериализует его в бинарный, Python-зависимый формат, используя протокол `pickle`, и сохраняет его в открытый для записи бинарный файл.

Приведенный ниже код:

```
import pickle
obj = {'Python': 1991, 'Java': 1995}
with open('file.pkl', 'wb') as file:
    pickle.dump(obj, file)
```

создает файл `file.pkl`, содержащий бинарное представление объекта `obj` на основе протокола `pickle`.

Функции `dump()` и `load()`

Функция `load()` принимает файловый объект, читает из него сериализованные данные, десериализует их в Python-объект и возвращает полученный Python-объект.

Приведенный ниже код:

```
import pickle
with open('file.pkl', 'rb') as file:
    obj = pickle.load(file)
    print(obj)
    print(type(obj))
```

ВЫВОДИТ:

```
{ 'Python': 1991, 'Java': 1995 }
<class 'dict'>
```

Функции dumps() и loads()

Функция **dumps()** выполняет сериализацию, как и функция `dump()`. Но вместо записи в бинарный файл, она возвращает эти сериализованные данные.

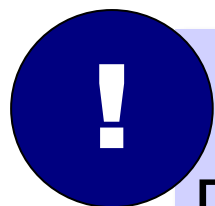
```
import pickle
obj = {'Python': 1991, 'Java': 1995}
binary_obj = pickle.dumps(obj)
print(binary_obj)
print(type(binary_obj))
```

ВЫВОДИТ:

```
b' \x80\x04\x95\x1b\x00\x00\x00\x00\x00\x00\x00
}\x94(\x8c\x06Python\x94M\xcb\x07\x8c\x04Java\x
\x94M\xcb\x07u.'
<class 'bytes'>
```

Функции `dumps()` и `loads()`

Поскольку протокол `pickle` использует бинарный формат данных, функция `dumps()` возвращает объект типа `bytes`.



Тип данных **`bytes`** — это неизменяемые последовательности отдельных байтов. Синтаксис для байтовых литералов в основном такой же, как и для строковых литералов, за исключением того, что добавляется префикс `b`.

Функции `dumps()` и `loads()`

Функция `loads()` выполняет десериализацию, как и функция `load()`. Но вместо того чтобы принимать файловый объект, она принимает объект типа `bytes`, содержащий сериализованные данные.

```
import pickle
obj = {'Python': 1991, 'Java': 1995}
binary_obj = pickle.dumps(obj)
new_obj = pickle.loads(binary_obj)
print(new_obj)
```

ВЫВОДИТ:

```
{ 'Python' : 1991, 'Java' : 1995 }
```

Функции `dumps()` и `loads()`



Что получим?

```
import pickle
obj = {'Java': 1995, 'C#': 2002}
binary_obj = pickle.dumps(obj)
new_obj = pickle.loads(binary_obj)
print(obj == new_obj)
print(obj is new_obj)
```

True

False



Объекты `obj` и `new_obj` равны, то есть имеют одинаковое содержимое, однако объекты не являются идентичными. Мы создали идеальную копию, но это всё же копия.

Примечания

Примечание 1. Модуль `pickle` может сериализовывать:

- ❑ все встроенные типы данных (`bool`, `int`, `float`, `complex`, `str`, `None`)
- ❑ списки, кортежи, словари и множества, содержащие любую комбинацию встроенных типов данных
- ❑ списки, кортежи, словари и множества, содержащие любую комбинацию списков, кортежей, словарей и множеств
- ❑ функции, классы и экземпляры классов

Примечания

Примечание 2. Модуль `pickle` может сериализовывать обычные функции (объявленные с помощью `def`), но не может сериализовывать лямбда-функции (объявленные с помощью `lambda`).

Примечание 3. Модуль `pickle` может сериализовывать функции из модуля `operator`.

Примечание 4. Модуль `pickle` не может сериализовывать генераторы.

Примечания

Примечание 5. Модули pickle и json делают практически одно и то же, по сути это одна и та же технология, только по-разному реализованная.

Примечание 6. Протокол pickle зависит от Python и не совместим с другими языками программирования. Если необходима совместимость с другими языками программирования, то следует использовать JSON сериализацию.

Примечания

Примечание 7. Протокол pickle — это бинарный формат данных. Убедитесь, что открываете файлы pickle в бинарном режиме, иначе данные при записи будут повреждены.

Примечание 8. Модуль pickle не защищен. Никогда не десериализуйте данные, полученные из ненадежного источника, так как они могут оказаться вредоносными и выполняющими произвольный код во время распаковки.

Примечание 9. Модуль pickle сериализует и десериализует данные быстрее чем модуль json.